PA3: Big Number Calculator Report

silver-kite-2a91

1. 大數相加由於資料範圍太大不能用整數資料型態表示，且整數陣列也有無法表達清楚的問題，所以解決方就是把數字放在 char 陣列中，一個一個拿出來放在個位、十位、百位、千位…

程式碼架構:

1

```
#include <iostream>
#include <string.h>
using namespace std;

int main()
{
    while(true)
    {
        cout << "----Welcome to big number calculator----" << endl;
        cout << "Please choose the function" << endl;
        cout << "1. Addition" << endl;
        cout << "2. Subtraction" << endl;
        cout << "3. Multiplication" << endl;
        cout << "0. Exit" << endl;
        cout << "-------------------------" << endl;
        int option;
        (cin >> option).get();
        if(option == 0)
        {
            cout << "Good bye!";
            return 0;
        }
        else if(option > 3 || option < 0)
        {
            cout << "Error! Please try again." << endl << endl;

        }
        else{}
```

1~28 行為系統歡迎畫面以及偵測錯誤函數功能輸入

30

```
        if(option == 1)
        {
            int a[5][101] = {0}, k = 0, l = 0, sum[3][102]={0};
            char al[3][101]={'\n'};
            cout<<"----Now for Addition----"<<endl;
            cout<<"Please enter two numbers.\n";
            cin.getline(al[1],101);
            cin.getline(al[2],101);
            for(int j = 1; j <= 2; j++)
            {
                for(int i = 0; i < 101; i++)
                {
                    a[j][i] = al[j][i] - '0';
                    switch(a[j][i])
                    {
                    case 0:
                        a[j][i + k] = 0;
                        break;
                    case 1:
                        a[j][i + k] = 1;
                        break;
                    case 2:
                        a[j][i + k] = 2;
                        break;
                    case 3:
                        a[j][i + k] = 3;
                        break;

                    case 4:
                        a[j][i + k] = 4;
                        break;
                    case 5:
                        a[j][i + k] = 5;
                        break;
                    case 6:
                        a[j][i + k] = 6;
                        break;
                    case 7:
                        a[j][i + k] = 7;
                        break;
                    case 8:
                        a[j][i + k] = 8;
                        break;
                    case 9:
                        a[j][i + k] = 9;
                        break;
                    default:
                        a[j][i + k] = 10;
                    }

                }
            }
```

80

30~80 行為將輸入的數字轉換成數字(關鍵程式碼在第 42 行)

81

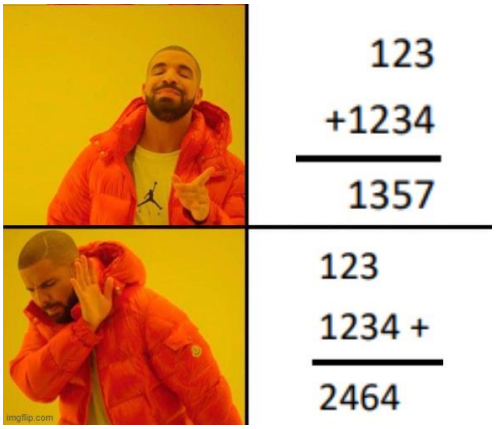
```
81          for(int j = 1; j <= 2; j++)
82          {
83              while(a[j][100 - k] == 10 && a[j][99 - k] == 10)
84              {
85                  k++;
86              }
87              k++;
88              for(int i = 0; i < 101 - k; i++)
89              {
90                  a[j + 2][k + i] = a[j][i];
91              }
92              k = 0;
93          }
94          for(int j = 1;j <= 4; j++)
95          {
96              for(int i = 0;i < 101;i++)
97              {
98                  if(a[j][i] == 10)
99                  {
100                     a[j][i]=0;
101                 }
102             }
103         }
104         for(int i = 0; i < 101; i++)
105         {
106             {
107                 sum[1][101 - i]=(a[3][100-i] + a[4][100-i] + sum[2][101-i]) % 10;
108                 sum[2][100 - i]=(a[3][100-i] + a[4][100-i] + sum[2][101-i]) / 10;
109             }
110         }
111         sum[1][0] = sum[2][0];
112         while(sum[1][1] == 0 && sum[1][1 + 1] == 0)
113         {
114             l++;
115         }
116         cout<<"Result: ";
117         if(l > 100) cout << "0";
118         for(int i = l + 1; i < 102; i++)
119         {
120             cout << sum[1][i];
121         }
122         cout << endl;
```

122

81~122 行為先平移再行計算加法

平移: (81~103 行)根據輸入 char[]的數字來看它會將數字由左而右填入數字，造成數字對但是單位不對。

例如: 輸入 123 後，我的 char a1[1][101]會將這三個數字放在 a1[1][0]、a1[1][1] 、a1[1][2]，再輸入 1234，我的 char a1[2][101]會將這三個數字放在 a1[2][0]、a1[2][1]、a1[2][2]、a1[2][3]，由於我們在傳統上做加減法時習慣向右對齊，造成計算上的不便。



上圖是向右對齊，是我們希望的結果
下圖是 char[]儲存進 int[]後會出現的結果
(因為 char[0]跟 int[0]都在左邊，所以存進去後數值會向左對齊，那就會使位數錯亂，相加出不對的結果)

所以要先對我們所輸入的數字進行平移的動作，也就是把**個位移到陣列編號最大**的地方。如何實行: 先用 strlen 讀入我們所輸入的 char 陣列長度(char 陣列長度是以一個字元為一個單位)，**再用 int 陣列長度扣掉該長度後，開始將 char 陣列的數字轉換(扣 '0')到 int 陣列，途中 strlen 會不斷扣 1，p 不斷加 1 讓數字平移至正確位數。舉例: 輸入 123，strlen(a1)就會是 3，a[10]陣列長度為 10，那 char a1[0] = 1 會填入 int a[10 – 3] = int a[7]，char a1[1] = 2 會填入 int a[8]，**

char a1[2] = 3 會填入 int a[9]。

下面是程式碼

```
int a[10] = {0};
char a1[11] = {'\n'};
cin.getline(a1, 11);
cout << strlen(a1) << endl;
for (int p = 0, s = strlen(a1); p < 10; p++)
{a[10 - s] = a1[p] - '0';
s--;}
for (int i = 0; i < 10; i++)
{cout << a[i]; }
```

原本是”1,2,3 ”
修改後” 1,2,3”
完成了平移，後續計算會方便許多，不然沒有平移後面會走更多路(目前沒時間，或許寒假可以來寫寫看)

104

122

```
104  for(int i = 0; i < 101; i++)
105  {
106      {
107          sum[1][101 - i]=(a[3][100-i] + a[4][100-i] + sum[2][101-i]) % 10;
108          sum[2][100 - i]=(a[3][100-i] + a[4][100-i] + sum[2][101-i]) / 10;
109      }
110  }
111  sum[1][0] = sum[2][0];
112  while(sum[1][1] == 0 && sum[1][1 + 1] == 0)
113  {
114      l++;
115  }
116  cout<<"Result: ";
117  if(l > 100) cout << "0";
118  for(int i = l + 1; i < 102; i++)
119  {
120      cout << sum[1][i];
121  }
122  cout << endl;
```

加法: (104~122行)將平移好的數字放從a[1][ ]跟a[2][ ]拿到a[3][ ]跟a[4][ ]，

然後依個、十、百、千、萬…分別去做相加，其中進位的 1 會放在 sum[2][ ]中的下一位(例如個位鄉加高過 10 就會在十位+1)，以此類推…。最後，sum[1][ ]會放置相加的總和，**透過從 sum[1][0]檢查至 sum[1][101]兩位不為 0 的地方開始印出加法的結果(因為印出的最高位不能為 0)**。

122

```
124  int b[5][101] = {0}, m = 0, n = 0, sum1[3][102] = {0}, mean[3][102] = {0};
125  for(int j = 1; j <= 2; j++)
126  {
127      for(int i = 0; i < 101; i++)
128      {
129          b[j][i] = a1[j][i] - '0';
130          switch(b[j][i])
131          {
132          case 0:
133              b[j][i + m]=0;
134              break;
135          case 1:
136              b[j][i + m] = 1;
137              break;
138          case 2:
139              b[j][i + m] = 2;
140              break;
141          case 3:
142              b[j][i + m] = 3;
143              break;
144          case 4:
145              b[j][i + m] = 4;
146              break;
147          case 5:
148              b[j][i + m] = 5;
149              break;
150          case 6:
```

```
                    b[j][i + m] = 6;
                    break;
                case 7:
                    b[j][i + m] = 7;
                    break;
                case 8:
                    b[j][i + m] = 8;
                    break;
                case 9:
                    b[j][i + m] = 9;
                    break;
                default:
                    b[j][i + m] = '\n';
                }
            }
        }
        for(int j = 1; j <= 2; j++)
        {
            while(b[j][100 - m] == '\n' && b[j][99 - m] == '\n')
            {
                m++;
            }
            m++;
            for(int i = 0; i < 101 - m; i++)
            {
                b[j + 2][m + i] = b[j][i];
            }

            m = 0;
        }
        for(int j = 1; j <= 4; j++)
        {
            for(int i = 0;i < 101; i++)
            {
                if(b[j][i] == '\n')
                {
                    b[j][i] = 0;
                }
            }
        }
        for(int i = 0; i < 101; i++)
        {
            {
                suml[1][101 - i] = (b[3][100 - i] + b[4][100 - i] + suml[2][101 - i]) % 10;
                suml[2][100 - i] = (b[3][100 - i] + b[4][100 - i] + suml[2][101 - i]) / 10;
            }
        }
        suml[1][0] = suml[2][0];
        while(suml[1][n] == 0 && suml[1][n+1] == 0)
        {
            n++;
        }
        for(int i = n + 1; i < 102; i++)
        {
            if((suml[1][i] + mean[2][i]) % 2 != 0 && i < 102)

            {
                mean[2][i + 1] += 10;
            }
            mean[1][i] = (suml[1][i] + mean[2][i]) / 2;
        }
        cout << "Mean: ";
        if(mean[1][n + 1] == 0)
        {
            n++;
        }
        if(n > 100) cout << "0";
        for(int i = n + 1; i < 102; i++)
        {
            cout << mean[1][i];
        }
        cout << endl << endl;
    }
```

221

平均: (124~221 行)先將各個的初始兩個要做運算的陣列(加法、減法、乘法)拿來做加法後從最大位 b[3][100]跟 b[4][100]開始，先除以 2 取整數部分，如果餘數不為 0 就給下面一位加上 10 後再除以 2 取整數部分。以 b[3][ ]跟 b[4][ ]相加結果為 184 存放在 sum[1][98]、sum[1][99]、sum[1][100]舉例，

- (1 + 10*0)/2 = 0
- (1 + 10*0)%2 = 1 (百位還有 1 尚未被用盡，所以在十位加上一個 10)
- (8 + 10*1)/2 = 9
- (8 + 10*1)%2 = 0 (十位已經除盡，但是為了整齊性，一律在個位加上 10 乘上該餘數)

- (4 + 10*0)/2 = 2
- 紅色數字為平均結果(放在 mean[1][])，褐色為原數字，藍色、綠色為進位至小一位(ex. 個位至十位)的數字(若該位是奇數)(放在 mean[2][]

最後結果放在 mean[1][ ]然後從較大位數開始依序輸出。

222

```
222         else if(option == 2)
223         {
224             int a[5][101] = {0}, k = 0, l = 0, sub[3][102] = {0};
225             char al[3][101]={'\n'};
226             cout << "----Now for Subtraction----" << endl;
227             cout << "Please enter two numbers.\n";
228             cin.getline(al[1], 101);
229             cin.getline(al[2], 101);
230             for(int j = 1; j <= 2; j++)
231             {
232                 for(int i = 0; i < 101; i++)
233                 {
234                     a[j][i] = al[j][i] - '0';
235                     switch(a[j][i])
236                     {
237                     case 0:
238                         a[j][i + k] = 0;
239                         break;
240                     case 1:
241                         a[j][i + k] = 1;
242                         break;
243                     case 2:
244                         a[j][i + k] = 2;
245                         break;
246                     case 3:
247                         a[j][i + k] = 3;
248                         break;
249                     case 4:
250                         a[j][i + k] = 4;
251                         break;
252                     case 5:
253                         a[j][i + k] = 5;
254                         break;
255                     case 6:
256                         a[j][i + k] = 6;
257                         break;
258                     case 7:
259                         a[j][i + k] = 7;
260                         break;
261                     case 8:
262                         a[j][i + k] = 8;
263                         break;
264                     case 9:
265                         a[j][i + k] = 9;
266                         break;
267                     default:
268                         a[j][i + k] = '\n';
269                     }
270                 }
271             }
272             for(int j = 1; j <= 2; j++)
273             {
274                 while(a[j][100-k] == '\n' && a[j][99-k] == '\n')
275                 {
276                     k++;
277                 }
278                 k++;
279                 for(int i = 0; i < 101 - k; i++)
280                 {
281                     a[j + 2][k + i] = a[j][i];
282                 }
283                 k=0;
284             }
285             for(int j = 1; j <= 4; j++)
286             {
287                 for(int i = 0;i < 101; i++)
288                 {
289                     if(a[j][i] == '\n')
290                     {
291                         a[j][i] = 0;
292                     }
293                 }
294             }
295
296             for(int i = 0; i < 101; i++)
297             {
298                 {
299                     if((a[3][100-i] - a[4][100-i] - sub[2][100-i]) < 0)
300                     {
301                         sub[1][100-i] = a[3][100-i] - a[4][100-i] - sub[2][100-i] + 10;
302                         sub[2][99-i]++;
303                     }
304                     else
305                     {
306                         sub[1][100-i] = a[3][100-i] - a[4][100-i] - sub[2][100-i];
307                     }
308
309                 }
310             }
311
312             while(sub[1][l] == 0 && sub[1][l+1] == 0)
313             {
314                 l++;
315             }
316             cout << "Result: ";
317             if(l > 100) cout << "0";
318             for(int i = l + 1; i < 101; i++)
319             {
320                 cout << sub[1][i];
321             }
322             cout << endl;
```

322

減法: (222~322 行)跟加法原理一樣，不同的地方在於兩個同位的位數相減時如果小於 0 需要向前一位借一個 10，所以減數跟被減數分別放在 a[3][ ]跟 a[4][ ]，然後依個、十、百、千、萬…分別去做相減，從個位開始看，不夠減的話就要跟十位借，那到時候十位相減時會多減 1，以此類推。

422

```
422         else if(option == 3)
423         {
424             cout<<"----Now for Multiplication----"<<endl;
425             const int k = 100;
426             int a[7][2*k] = {{0}}, h = 0, m = 0;
427             char b[3][2*k] = {'a'}, bl[3][101] = {'a'};
428             cout << "Please enter two numbers\n";
429             cin.getline(b[1],101);
430             cin.getline(b[2],101);
431             for(int j = 1; j <= 2; j++)
432             {
433                 for(int i = 0; i < 2*k; i++)
434                 {
435                     a[j][i] = b[j][i] - '0';
436                     bl[j][i] = b[j][i];
437                     switch(a[j][i])
438                     {
439                     case 0:
440                         a[j][i + h] = 0;
441                         break;
442                     case 1:
443                         a[j][i + h] = 1;
444                         break;
445                     case 2:
446                         a[j][i + h] = 2;
447                         break;
448                     case 3:
449                         a[j][i + h] = 3;
450                         break;
451                     case 4:
452                         a[j][i + h] = 4;
453                         break;
454                     case 5:
455                         a[j][i + h] = 5;
456                         break;
457                     case 6:
458                         a[j][i + h] = 6;
459                         break;
460                     case 7:
461                         a[j][i + h] = 7;
462                         break;
463                     case 8:
464                         a[j][i + h] = 8;
465                         break;
466                     case 9:
467                         a[j][i + h] = 9;
468                         break;
469                     default:
470                         a[j][i + h] = 10;
471                     }
472                 }
473             }
474             for(int j = 1; j <= 2; j++)
475             {
476                 while(a[j][2*k - 1 - h] == 10 && a[j][2*k - 2 - h] == 10)
477                 {
478                     h++;
479                 }
480                 h++;
481                 for(int i = 0; i < 2*k - h; i++)
482                 {
483                     a[j + 2][h + i]=a[j][i];
484                 }
485                 h = 0;
486             }
487             for(int j = 1; j <= 4; j++)
488             {
489                 for(int i = 0; i < 2*k; i++)
490                 {
491                     if(a[j][i] == 10)
492                     {
493                         a[j][i] = 0;
494                     }
495                 }
496             }
497             for (int i = 0; i < k; i++)
498             {
499                 for (int j = 0; j < 2*k;j++)
500                 {
501                     a[5][2*k - 1 - j] = a[3][2*k - 1 - j]*a[4][2*k-  1 - i] + a[5][2*k - 1 - j];
```

```
502                 }
503                 for(int l = 0; l < 2*k; l++)
504                 {
505                     a[3][l] = a[3][l + 1];
506                 }
507                 a[3][2*k - 1]=0;
508             }
509             for(int i = 0; i < 2*k; i++)
510             {
511                 if(2*k-2 - i < 0);
512                 else
513                 {
514                     a[6][2*k - 2 - i] = (a[5][2*k - 1 - i] + a[6][2*k - 1 - i])/10;
515                 }
516                 a[5][2*k-1-i] = (a[5][2*k-1-i] + a[6][2*k-1-i])%10;
517             }
518             while(a[5][m] == 0 && a[5][m+1] == 0)
519             {
520                 m++;
521             }
522             cout << "Result: ";
523             if(m == 399)
524             {
525                 cout << "0";
526             }
527             for(int i = m + 1; i < 2*k; i++)
528
529             {
530                 cout << a[5][i];
531             }
532             cout << endl;
```

乘法: (422~532 行)跟我們用手算乘法一模一樣，以 37*73 為例，

$$\begin{bmatrix} & & 3 & 7 \\ \times & & 7 & 3 \\ & & 9 & 21 \\ 0 & 21 & 49 & 0 \\ 0 & 21 & 58 & 21 \\ 0 & 7 & 0 & 1 \\ 2 & 6 & 2 & 0 \\ 2 & 7 & 0 & 1 \end{bmatrix}$$

儲存在 a[1]

儲存在 a[2]

儲存在 a[3]

儲存在 a[4]

a[6](510~518 行)將進位元加到下一位

a[5]印出每一位 mod10 的部份，最後加上 a[6]

先算出各位數相乘的結果(先無須進位)，再取 mod 10 大於 10 的部分進到下一位，中間**減法(323~421 行)跟乘法的平均(534~633 行)由於跟加法平均程式碼相同，故不重覆貼上**。

2. PA3 公開的測資皆可以得到預期輸出。

3. 難處是看到大數相加一開始完全沒有想法，後來是剩下兩周時老師給了提示方得以拼湊出使用 char 陣列儲存數字的方法。因為必須先 figure out 實現一個一個數字提取的方法後才能往下走，所以下一個難處就是平移問題，後續的陣列加減乘法只是要將位數對齊而已。

4. 沒有任何 bonus function。負數以及除法、餘數由於時間上的關係未能完成。

5. 還是不習慣 PBL 的教學方法，先丟問題在給 tools 後通常已經過了一半時間，所以都會寫得很趕，解決方法就是: 自學 + 預習。